

Parte 1

4	Explica el algoritmo correspondiente para cada atributo. No es necesario calcular su complejidad exacta, pero sí describir y justificar cómo se comporta. • Para los atributos numéricos, puede emplearse cualquier algoritmo con complejidad $O(n \log n)$, como mergesort o heapsort. • Para ordenar por tag, lo más eficiente es radixsort, que permite un ordenamiento lineal. Evaluación: 2 pts por ítem correcto. 1 pt si hay errores menores. 0 pts si la propuesta no cumple la complejidad esperada.
2	Lo esperado es utilizar insertion sort. Dado que sólo se insertan unos pocos elementos no ordenados en un conjunto mayor ya ordenado, el algoritmo opera en su mejor caso, logrando una complejidad lineal $O(n)$, donde n es el tamaño del arreglo. • 2 pts: Usa insertion sort y justifica correctamente por qué es lineal. • 1 pt: Usa insertion sort pero no justifica adecuadamente. • 0 pts: Propone un algoritmo que no cumple la complejidad.

Importante: *Se otorgará puntaje completo si se emplea un algoritmo distinto al indicado, siempre que cumpla con la complejidad requerida.*

Parte 2

4	Explica las estructuras de datos utilizadas y justifica su elección en función de la eficiencia. • Para búsquedas según un único atributo, se espera el uso de un AVL, que garantiza operaciones en $O(\log n)$. Un ABB no balanceado podría degradar a $O(n)$ en el peor caso. Para búsquedas en rango, el mismo AVL puede recorrer los elementos en orden e imprimirlos a medida que se visitan. • Para búsquedas por dos atributos (speed y delay), puede emplearse un AVL bidimensional: un árbol principal ordenado por el primer atributo, y en cada nodo otro AVL que almacene los valores asociados al segundo. • Para el evento SIMILAR-RANGE, se puede construir un AVL que mantenga en cada nodo la distancia mínima y las referencias a los pares de nodos correspondientes. • En el caso de los chips, se recomienda utilizar Treaps, que permiten operaciones de búsqueda, split y merge en complejidad promedio $O(\log K)$, siendo K la cantidad de chips. Evaluación: 1 pt por estructura correctamente elegida y justificada. 0.5 pts si la justificación es parcial. 0 pts si no cumple con la complejidad pedida.
2	Los Treaps tienen un balanceo más flexible que los árboles vistos en clases. Aunque no están perfectamente balanceados, reducen la cantidad de rotaciones necesarias para operaciones de merge y split, resultando más eficientes que un AVL en estos casos. • 2 pts: Menciona todo lo anterior. • 1 pt: Describe complejidades del Treap pero no compara con AVL. • 0 pts: En otro caso.

Importante: Se otorgará puntaje completo si se usan estructuras distintas, siempre que cumplan con la complejidad requerida.